MICROPROCESSORS PROJECT REPORT (Gamepad Controller System)

Group Members:
Mustafa Haki KARACA 61230001 EEE
Kerim GÜLER 61230005 EEE
Github: https://github.com/kerimguler/PIC16F877A-Gamepad-Controller

---

1. Introduction

In this experiment, a Gamepad Controller System was designed using the PIC16F877A microcontroller. Usually microprocessors need extra external circuitry to make it work but this microcontroller is different. In embedded system development it is always needed to communicate with outside world. That's why General Purpose Input/Output (GPIO) pins are used. These pins have a crucial role because they can be configured as input or output to make the circuit work. For example they can be used to read analog joysticks and push buttons.

Microcontrollers are used in many electronic systems to read inputs and control outputs. In this project, reading inputs from switches is not enough. The microcontroller needs to send data to a computer. Serial communication is used for this part. After reading the hardware, a Python program translates the signals to play a PC game.

The system also includes an MPU6050 sensor. But Proteus does not have a real gyroscope module. To test the I2C communication, 4 empty headers and an I2C debugger were used instead of the sensor. When the communication is successful, the debugger shows an FF value. The main goal of this project is to understand GPIO operation and verify the communication interfaces using both hardware implementation and Proteus simulation.

2. Objective

The objective of this experiment is to understand how GPIO pins work on the PIC16F877A microcontroller. In this lab, the pins are used as input and output to see how the microcontroller works with other components. Actually they must be configured as input or output first to make it work. After that, data from input pins can be read when a user moves a joystick or presses a switch.

Another objective is to read these analog values, and send them to a computer. With this, it is possible to see how the system reacts with a Python script.

The operation of I2C pins in a circuit needs to be seen. So the objective is getting familiar with this communication and testing it in simulation. Because there is no real sensor in Proteus, reading the I2C debugger response is intended.

3. Materials and Equipment

Hardware Components

- PIC16F877A Microcontroller
- HW-504 / KY-023 Analog Joystick Modules (2x)
- MPU6050 Accelerometer and Gyroscope Module
- Push Buttons
- Vibration Motor
- LM324 Operational Amplifier (DIP-14)
- Resistors (1 kΩ, 4.7 kΩ, 10 kΩ, 100 kΩ)
- Capacitors (1 nF, 10 nF, 100 nF, 1 µF, 10 µF)
- Breadboard
- Jumper Wires
- USB Cable
- 5V Power Supply
- PC817 Optocoupler

Development and Testing Equipment

- Computer / Laptop
- MPLAB X IDE
- XC8 Compiler
- Python 3.x
- PySerial Library
- vgamepad Library

Software Tools

- MPLAB X IDE
- XC8 Compiler
- Python
- PySerial
- vgamepad
- Serial Monitor Software

4. Procedure

First, the circuit was prepared in Proteus. The PIC16F877A microcontroller was placed on the workspace and the required components were added. Two joystick modules, push buttons, an MPU6050 sensor and a vibration motor were connected to the microcontroller. The crystal oscillator and other required connections were also completed before starting the simulation.

After finishing the circuit, the code was written in MPLAB X IDE using C language. The analog pins were configured for joystick and trigger inputs. Button pins were configured as digital inputs. UART communication was added to transfer the controller data to the computer. I2C communication was also configured for the MPU6050 sensor.

After the code was completed, it was compiled and a HEX file was generated. The generated HEX file was first tested in Proteus and then programmed into the PIC16F877A hardware. The simulation was then started and the outputs were checked from the Virtual Terminal.

At the beginning, some values were not displayed as expected. Because of this, the ADC settings and pin connections were checked again. After a few changes, the joystick values started to appear correctly on the terminal window. The MPU6050 readings were also tested and the transmitted values were observed during the simulation.

After confirming that the serial communication was working, a Python application was used on the computer side. The received data was converted into a virtual Xbox controller. Different joystick movements, button presses and trigger values were tested several times. The response of the virtual controller was observed during these tests.

Finally, the vibration motor feature was checked. When force feedback was generated by the virtual controller, a command was sent back through the serial connection and the motor response was verified. A PC817 optocoupler was used in the motor control circuit to provide electrical isolation between the microcontroller and the motor driver stage. After testing different input combinations, the system operated as expected and the required outputs were obtained.

5. Circuit Diagram

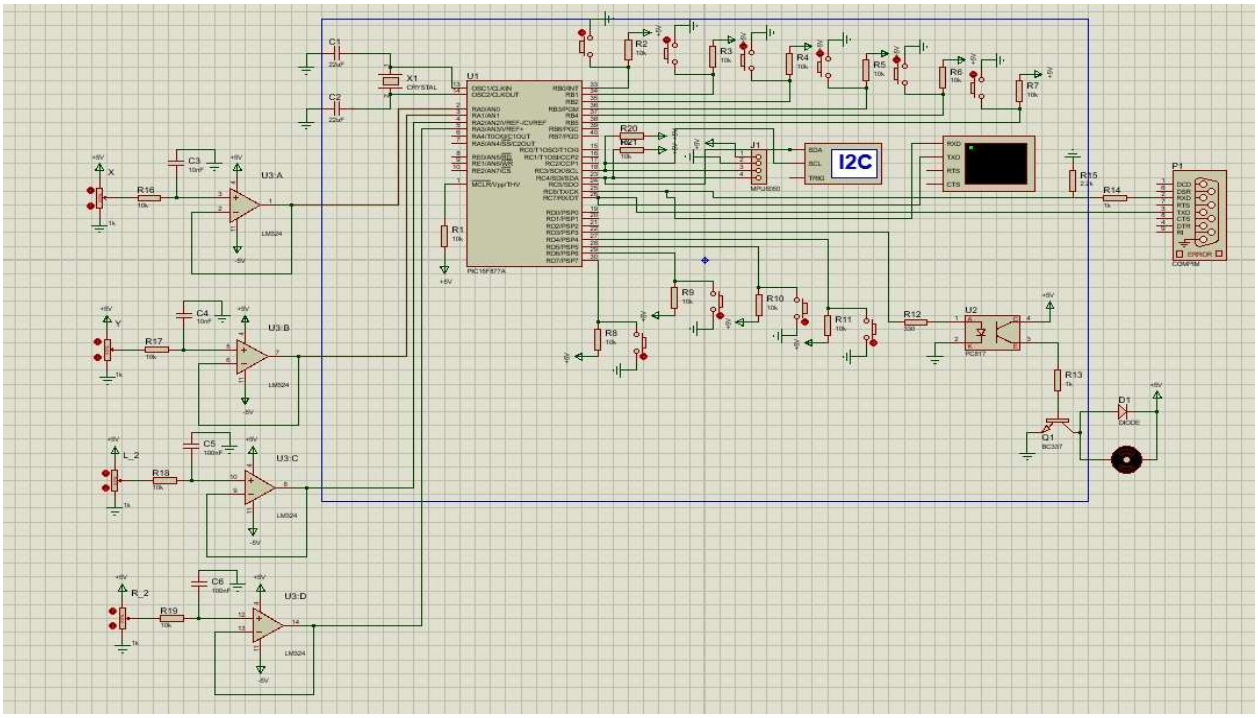

Figure 1. Complete Proteus circuit design of the Gamepad Controller System including the PIC16F877A microcontroller, joystick inputs, MPU6050 sensor, push buttons, UART interface, and vibration motor control circuit.

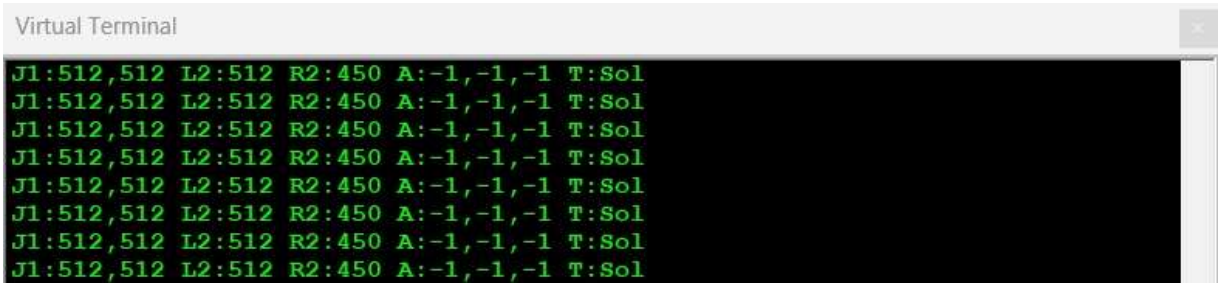

Figure 2. Virtual Terminal output showing the transmitted joystick, trigger, accelerometer, and button data during simulation.

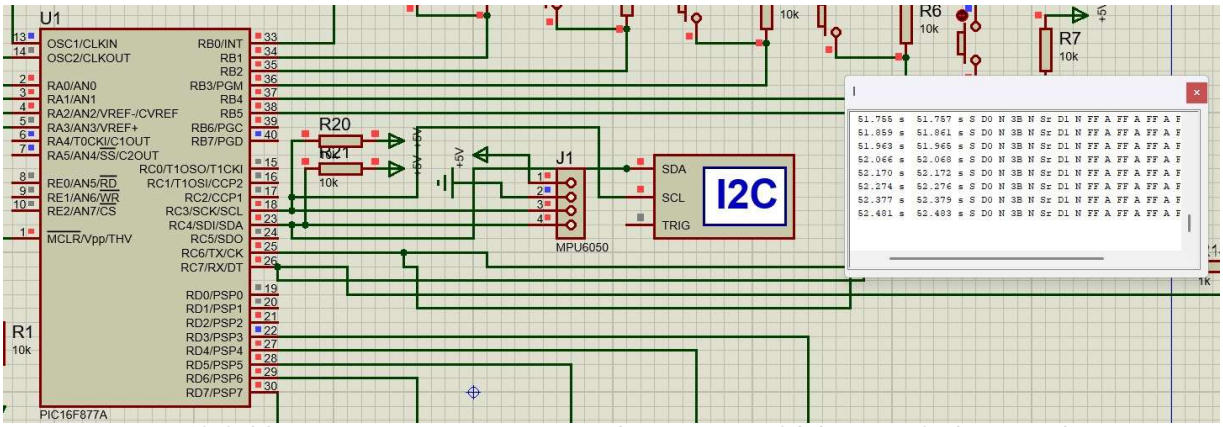

Figure 3. MPU6050 I2C communication test in Proteus showing successful data transfer between the PIC16F877A and the sensor module.

6. Code

The software part of the project consists of two sections. The first section is the firmware running on the PIC16F877A microcontroller. This code reads the joystick positions, trigger values, button states, and MPU6050 sensor data. The collected information is then transmitted to the computer through UART communication.

The second section is a Python application running on the computer. This program receives the serial data, processes the incoming values, and maps them to a virtual Xbox controller using the vgamepad library. It also supports vibration feedback by sending motor control commands back to the microcontroller when required.

Figure 4. PIC16F877A source code written in MPLAB X IDE for reading joystick, trigger, button, and MPU6050 sensor data and transmitting the information through UART.

```
/*******************************************************************************
 * Project Name : Gamepad Controller System
 *
 * Course       : Electrical and Electronics Engineering (EEE)
 *
 * Students:
 *   Mustafa Haki Karaca
 *   Student ID : 61230001
 *   Department : EEE
 *
 *   Kerim Güler
 *   Student ID : 61230005
 *   Department : EEE
 *
 * Description:
 * This project reads joystick, trigger, button, and MPU6050 sensor data,
 * sends the data through UART, and controls a vibration motor.
 ******************************************************************************/

#include <xc.h>
#include <stdio.h>
#include <stdint.h>
```

```
#define _XTAL_FREQ 20000000

#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config BOREN = ON
#pragma config LVP = OFF
#pragma config CPD = OFF
#pragma config WRT = OFF
#pragma config CP = OFF

#define MOTOR_PIN PORTDbits.RD3  // Vibration motor output pin

// Send a single character over UART
void UART_Write(char data) {
   while(!TRMT);
   TXREG = data;
}

// Send a string over UART
void UART_Write_String(const char *text) {
   for(int i = 0; text[i] != '\0'; i++) {
      UART_Write(text[i]);
   }
}

// Read data from the selected ADC channel
uint16_t ADC_Read(uint8_t channel) {
   ADCON0 &= 0xC5;
   ADCON0 |= (channel << 3);
   __delay_ms(2);
   GO_nDONE = 1;
   while(GO_nDONE);
   return ((ADRESH << 8) + ADRESL);
}

// Initialize I2C module
void I2C_Init(const unsigned long clock) {
   SSPCON = 0x28;
   SSPCON2 = 0x00;
```

```
    SSPADD = (_XTAL_FREQ / (4 * clock)) - 1;
    SSPSTAT = 0x00;
    TRISCbits.TRISC3 = 1;
    TRISCbits.TRISC4 = 1;
}

// Wait until I2C bus is free
void I2C_Wait() {
    uint16_t timeout = 0;
    while (((SSPCON2 & 0x1F) || (SSPSTAT & 0x04)) && (timeout < 1000)) {
        timeout++;
        __delay_us(1);
    }
}

// Start I2C communication
void I2C_Start() {
    I2C_Wait();
    SEN = 1;
}

// Generate repeated start
void I2C_RepeatedStart() {
    I2C_Wait();
    RSEN = 1;
}

// Stop I2C communication
void I2C_Stop() {
    I2C_Wait();
    PEN = 1;
}

// Send one byte over I2C
void I2C_Write(uint8_t data) {
    I2C_Wait();
    SSPBUF = data;
}

// Read one byte from I2C
```

```
uint8_t I2C_Read(uint8_t ack) {
    uint8_t temp;
    I2C_Wait();
    RCEN = 1;
    I2C_Wait();
    temp = SSPBUF;
    I2C_Wait();
    ACKDT = (ack) ? 0 : 1;
    ACKEN = 1;
    return temp;
}

// Initialize MPU6050 sensor
void MPU6050_Init() {
    I2C_Start();
    I2C_Write(0xD0);
    I2C_Write(0x6B);
    I2C_Write(0x00);
    I2C_Stop();
}

void main(void) {

    // Configure I/O directions
    TRISA = 0x0F;
    TRISE = 0x03;
    TRISB = 0x3F;
    TRISD = 0xF3;

    // Turn motor off at startup
    MOTOR_PIN = 0;

    // Configure ADC
    ADCON1 = 0x80;
    ADCON0 = 0x41;

    // Configure UART
    TRISCbits.TRISC6 = 0;
    TRISCbits.TRISC7 = 1;
    SPBRG = 129;
```

```
TXSTA = 0x24;
RCSTA = 0x90;

// Initialize I2C and MPU6050
I2C_Init(10000);
__delay_ms(100);
MPU6050_Init();

char buffer[40];
uint16_t j1_x, j1_y, j2_x, j2_y, l2_trig, r2_trig;
int16_t ax, ay, az;

while(1) {

    // Read joystick and trigger values
    j1_x    = ADC_Read(2);
    j1_y    = ADC_Read(3);
    j2_x    = ADC_Read(0);
    j2_y    = ADC_Read(1);
    l2_trig = ADC_Read(6);
    r2_trig = ADC_Read(5);

    // Read accelerometer data from MPU6050
    I2C_Start();
    I2C_Write(0xD0);
    I2C_Write(0x3B);
    I2C_RepeatedStart();
    I2C_Write(0xD1);
    ax = (I2C_Read(1) << 8) | I2C_Read(1);
    ay = (I2C_Read(1) << 8) | I2C_Read(1);
    az = (I2C_Read(1) << 8) | I2C_Read(0);
    I2C_Stop();

    // Send joystick values
    sprintf(buffer, "J1:%u,%u J2:%u,%u ", j1_x, j1_y, j2_x, j2_y);
    UART_Write_String(buffer);

    // Send trigger values
    sprintf(buffer, "L2:%u R2:%u ", l2_trig, r2_trig);
    UART_Write_String(buffer);
```

```
        // Send accelerometer values
        sprintf(buffer, "A:%d,%d,%d T:", ax, ay, az);
        UART_Write_String(buffer);

        // Check pressed buttons
        if(PORTBbits.RB0 == 0) UART_Write_String("Sag ");
        if(PORTBbits.RB1 == 0) UART_Write_String("RB ");
        if(PORTBbits.RB2 == 0) UART_Write_String("X ");
        if(PORTBbits.RB3 == 0) UART_Write_String("B ");
        if(PORTBbits.RB4 == 0) UART_Write_String("A ");
        if(PORTBbits.RB5 == 0) UART_Write_String("Y ");
        if(PORTDbits.RD4 == 0) UART_Write_String("Sol ");
        if(PORTDbits.RD5 == 0) UART_Write_String("Yukari ");
        if(PORTDbits.RD6 == 0) UART_Write_String("Asagi ");
        if(PORTDbits.RD7 == 0) UART_Write_String("LB ");

        // End current message
        UART_Write_String("\r\n");

        // Receive motor control command
        if(RCIF) {
            char rx_data = RCREG;
            if(rx_data == 'M') MOTOR_PIN = 1;
            else if (rx_data == 'S') MOTOR_PIN = 0;
        }

        // Small update delay
        __delay_ms(50);
    }
}
```

Figure 5. Python source code used to receive serial data from the microcontroller and convert it into a virtual Xbox controller using the vgamepad library.

```python
############################################################
#################
# Project Name : Gamepad Controller System
#
# Course       : Electrical and Electronics Engineering (EEE)
#
# Students:
#   Mustafa Haki Karaca
#   Student ID : 61230001
#   Department : EEE
#
#   Kerim Güler
#   Student ID : 61230005
#   Department : EEE
#
# Description:
# This program receives controller data from a serial port, maps it to a
# virtual Xbox controller, and controls rumble feedback.
############################################################
#################

import serial
import vgamepad as vg
import time

# Serial communication settings
SERIAL_PORT = 'COM14'
BAUD_RATE = 9600

ser = None

# Axis inversion settings
INVERT_J1_X = True
```

```
INVERT_J1_Y = True

INVERT_J2_X = False
INVERT_J2_Y = True

# Mapping between received button names and Xbox buttons
BUTTON_MAP = {
    "X": vg.XUSB_BUTTON.XUSB_GAMEPAD_X,
    "Y": vg.XUSB_BUTTON.XUSB_GAMEPAD_Y,
    "B": vg.XUSB_BUTTON.XUSB_GAMEPAD_B,
    "A": vg.XUSB_BUTTON.XUSB_GAMEPAD_A,
    "RB": vg.XUSB_BUTTON.XUSB_GAMEPAD_RIGHT_SHOULDER,
    "LB": vg.XUSB_BUTTON.XUSB_GAMEPAD_LEFT_SHOULDER,
    "Yukari": vg.XUSB_BUTTON.XUSB_GAMEPAD_DPAD_UP,
    "Asagi": vg.XUSB_BUTTON.XUSB_GAMEPAD_DPAD_DOWN,
    "Sag": vg.XUSB_BUTTON.XUSB_GAMEPAD_DPAD_RIGHT,
    "Sol": vg.XUSB_BUTTON.XUSB_GAMEPAD_DPAD_LEFT
}

# Handle rumble feedback from the virtual controller
def rumble_callback(client, target, large_motor, small_motor, led_number,
user_data):
    global ser
    try:
        if large_motor > 0 or small_motor > 0:
            if ser and ser.is_open: ser.write(b'M')
        else:
            if ser and ser.is_open: ser.write(b'S')
    except Exception:
        pass

# Convert joystick ADC values to Xbox joystick range
def map_analog_custom(val, center=512, min_val=0, max_val=750,
deadzone=30):
    if val > max_val: val = max_val
    if val < min_val: val = min_val

    if abs(val - center) <= deadzone:
```

```
        return 0

    if val > center:
        return int(((val - (center + deadzone)) / (max_val - (center +
deadzone))) * 32767)
    else:
        return int(((val - min_val) / ((center - deadzone) - min_val)) * 32768 -
32768)

# Convert trigger ADC values to Xbox trigger range
def map_trigger_custom(val, deadzone=80, max_val=750):
    if val < deadzone:
        return 0
    if val > max_val:
        return 255

    return int(((val - deadzone) / (max_val - deadzone)) * 255)

# Main program
def main():
    global ser

    # Create virtual Xbox controller
    gamepad = vg.VX360Gamepad()
    gamepad.register_notification(callback_function=rumble_callback)

    print("Virtual Xbox Controller Launched.")

    # Open serial connection
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=0.1)
    except Exception as e:
        print(f"Port Hatası: {e}")
        return

    while True:
        try:
            # Read one line from serial port
```

```
line = ser.readline().decode('utf-8').strip()
if not line:
    continue

line = line.replace("T:", "T: ")
gamepad.reset()

parts = line.split()
active_buttons = []

# Parse incoming controller data
for part in parts:

    # Process left joystick
    if part.startswith("J1:"):
        x, y = map(int, part[3:].split(','))
        j1_x = map_analog_custom(x)
        j1_y = map_analog_custom(y)
        if INVERT_J1_X: j1_x = -j1_x
        if INVERT_J1_Y: j1_y = -j1_y
        gamepad.left_joystick(x_value=j1_x, y_value=j1_y)

    # Process right joystick
    elif part.startswith("J2:"):
        x, y = map(int, part[3:].split(','))
        j2_x = map_analog_custom(x)
        j2_y = map_analog_custom(y)
        if INVERT_J2_X: j2_x = -j2_x
        if INVERT_J2_Y: j2_y = -j2_y
        gamepad.right_joystick(x_value=j2_x, y_value=j2_y)

    # Process left trigger
    elif part.startswith("L2:"):
        val = int(part[3:])
        gamepad.left_trigger(value=map_trigger_custom(val))

    # Process right trigger
    elif part.startswith("R2:"):
```

```
                val = int(part[3:])
                gamepad.right_trigger(value=map_trigger_custom(val))

            # Store active buttons
            elif part in BUTTON_MAP:
                active_buttons.append(part)

        # Update button states
        for btn_name, btn_code in BUTTON_MAP.items():
            if btn_name in active_buttons:
                gamepad.press_button(button=btn_code)
            else:
                gamepad.release_button(button=btn_code)

        # Send updated state to virtual controller
        gamepad.update()

    except KeyboardInterrupt:
        break
    except Exception:
        pass

# Program entry point
if __name__ == "__main__":
    main()
```

7. Results

The project was tested in Proteus after loading the HEX file into the PIC16F877A. During the simulation, joystick values, trigger values and button states were displayed on the Virtual Terminal.

The joysticks were tested first. When the joystick position changed, different values appeared on the terminal. The trigger values also changed according to their position.

The buttons were tested one by one. Their names appeared on the terminal when they were pressed. The displayed outputs matched the button inputs.

The MPU6050 sensor was also checked. Sensor values could be seen together with the joystick data. This showed that the I2C communication was working.

Some settings were checked again during the tests because a few values were not displayed correctly at first. After fixing these settings, the outputs became stable.

The Python application received the serial data and converted it into a virtual Xbox controller. The controller responded to joystick movements, trigger actions and button presses.

After completing the tests, the system worked correctly and the expected outputs were obtained. After successful simulation tests, the system was implemented on hardware. Similar results were obtained during hardware testing, and the virtual controller responded correctly to joystick and button inputs.

8. Discussion

The project worked according to the requirements after the tests were completed. The joystick values, trigger values and button inputs were read by the PIC16F877A and transmitted through UART communication. The values could be observed on the Virtual Terminal during the simulation.

At the beginning, some parts of the project did not work correctly. The transmitted data was not always displayed as expected, so the circuit connections and communication settings were checked again. After correcting a few mistakes, the outputs became more stable.

The MPU6050 sensor was also tested during the simulation. The sensor values were transmitted together with the other controller data. The communication between the sensor and the microcontroller worked without any major problem.

The Python application was tested after the hardware side was completed. Different joystick movements and button presses were used during the tests. After some adjustments, the virtual controller responded correctly to the received data.

This project was different from the previous experiments because several

components were used together in the same system. It was useful to see how analog inputs, sensors and serial communication can be combined in a single application. The motor control section also included a PC817 optocoupler to provide signal isolation between the microcontroller and the motor circuit.

9. Conclusion

In this project, a gamepad controller system was designed using the PIC16F877A microcontroller. Joysticks, buttons and the MPU6050 sensor were connected to the system and tested in simulation.

The transmitted data could be observed on the Virtual Terminal during the tests. The Python application was also able to receive this data and use it to control a virtual Xbox controller.

Some problems were encountered during the development stage, especially while testing the communication settings. After checking the connections and making a few changes, the system started to work correctly.

Overall, the project was completed successfully and the required outputs were obtained from all parts of the system.

10. References

[1] Microchip Technology Inc., PIC16F877A Data Sheet, Chandler, AZ, USA, 2003.

[2] InvenSense, MPU-6000 and MPU-6050 Product Specification, Revision 3.4, 2013.

[3] Microchip Technology Inc., MPLAB X IDE User's Guide, Chandler, AZ, USA.

[4] Y. Bouteiller, "vgamepad: Virtual Gamepad Library for Python," GitHub Repository. [Online]. Available: https://github.com/yannbouteiller/vgamepad